

Complex SQL Practice Questions

1. Relational Schema Setup (Design Challenge)

Design a University Database with the following:

- Students(student_id, name, email, department_id)
- Departments(department_id, department_name)
- Courses(course_id, course_name, department_id)
- Enrollments(enrollment_id, student_id, course_id, grade)
- Professors(professor_id, name, course_id)

Define all primary keys, foreign keys, and ensure that:

- A student can enroll in multiple courses.
- Each course belongs to one department.
- Each course can have only one professor.

2. Advanced ALTER with Constraints

- Add a CHECK constraint on Enrollments.grade so that it can only be 'A', 'B', 'C', 'D', 'F'.
- Add a UNIQUE constraint on Courses.course_name.
- Modify Students.email to be NOT NULL only if no null values exist.

3. DELETE and ON DELETE CASCADE

- Add ON DELETE CASCADE to Enrollments so if a student is deleted, their enrollments are too.
- Delete a department only if no students or courses are associated with it.

4. Nested UPDATE and Conditional Logic

- Update grades to 'Probation' if student has >3 enrollments and avg grade is 'C' or lower.
- Add column status (Active/Inactive) to Students based on enrollment in current semester.

5. Composite Foreign Keys and Constraints

- Create Payments(student_id, course_id, ...) where (student_id, course_id) references Enrollments.

- Enforce that a professor cannot teach two courses with the same name in the same semester.

6. Tricky INSERT Scenarios

- Insert student only if email does not exist (use NOT EXISTS).
- Insert multiple courses in one query and ignore duplicates based on course_name.

7. Data Audit & Validation (Subqueries + Joins)

- Find students enrolled in all CS department courses.
- List all courses not enrolled by any student this semester.
- List students who enrolled in more than 2 courses taught by the same professor.